COSC4606 - 001

Project 2 – Part 2

Manav Patel

239466460

Manav Patel
239466460

Cosc4606001
Project 2 – Part 2

Introduction:

The objective of this project is to transform the database prototype developed in Project 1 into a functioning application using modern software stack. Project 1 involved designing and building a Microsoft Access-based database system for managing a retail bicycle shop. The system included support for core operations such as customer and product management, sales tracking, inventory updates, repair services, and various business reports. While this prototype was suitable for initial development and learning purpose, I faced issues due to lack of scalability, security, and deployment flexibility required for a CRUD application.

In this second phase, the database system is transitioned into a two-tier client-server architecture using Java (Spring Boot), Microsoft SQL Server, and MySQL. The new system maintains all functional requirements from the original prototype while providing an additional secure user authentication and access control features. The use of JDBC (Java Database Connectivity) helps in communication between the Java application layer and the relational database backends.
The chosen architecture for this implementation is the two-tier model, where the client application communicates directly with the database server using JDBC. This approach was selected for its simplicity and ease of deployment, particularly for a standalone application on a local server where direct database access is feasible.

This implementation provides more flexibility than access as we can design and scale according to client's needs.

Setting Up the Environment:

Software Requirements:
1. Java 17 or above
2. MySQL (Add MySQL Workbench during installation)
3. MS SQL Server (Add SSMS and SQL server Configuration Manager)
4. An IDE (preferably Visual Studio Code)

For MySQL:

1. Open a folder attached with this report, named 'Trent-Bicycle-Shop-MySQL', in VS code or your choice of IDE.
2. Open MySQL workbench.
3. Open TrentBicycleShop.sql file in mySQL workbench. File Path is from: Trent-Bicycle-Shop-MySQL->src->main->resources->db_setup_queries->mysql->TrentBicycleShop.sql
4. Run the whole script.
5. Go back to your IDE and in your resources folder, you will find 'application.properties' where you must change the username and password.

```
spring.datasource.username=yourusername
spring.datasource.password=yourpw
```

6. Run the application from terminal at the root folder which is 'Trent-Bicycle-Shop-MySQL' using:

./mvnw spring-boot:run

Or

You can press any button that runs the program in your respective IDE.

For MS SQL Server:

1. Go to SQL server Configuration Manager -> SQL Server Network Configuration -> Protocols for MSSQLSERVER -> **Enable TCP/IP**
2. Now, open a folder attached with this report named 'Trent-Bicycle-Shop-MS-SQL', in VS code or your choice of IDE.
3. Open MySQL workbench.
4. Open MSSQL_Query.sql file in MS SQL Server Management Studio. File Path is from: Trent-Bicycle-Shop-MS-SQL ->src->main->resources->db_setup_queries->mysql-> MSSQL_Query.sql
5. Run the whole script.
6. Go back to your IDE and in your resources folder, you will find 'application.properties' where you must change the username and password.

spring.datasource.username=yourusername
spring.datasource.password=yourpw

7. Additionally, make sure the use you add here does exist in SSMS with permissions to **connect SQL**. If not, then on the left in SSMS, under **Object Explorer**, right click on **your server** and go to **properties**. In properties, go to **permissions** and select the **username** you chose in 'application.properties' and grant them permission to **'Connect SQL'**.
8. Run the application from terminal at the root folder which is 'Trent-Bicycle-Shop-MySQL' using:

./mvnw spring-boot:run

Or

You can press any button that runs the program in your respective IDE.

If your run is successful, you will see this in terminal:
[TimeStamp] INFO 62647 --- [trentbicycleshop] [restartedMain] c.m.t.TrentBicycleshopApplication : Started TrentBicycleshopApplication in 2.143 seconds (process running for 2.367)

Once you are successful, go to **http://localhost:8080/login**

Next steps will be in Navigation section of this report describing how to use the application.

Database Designs:

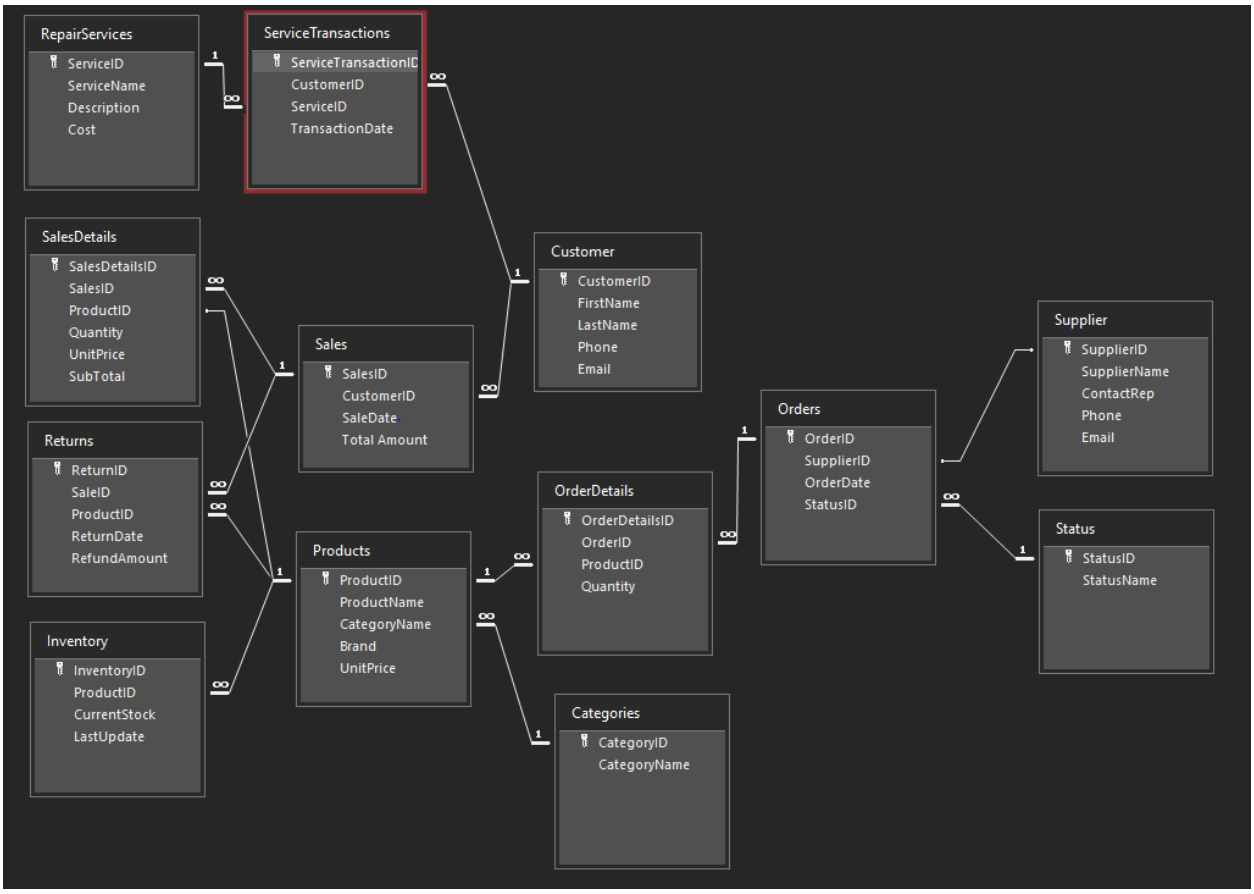


Figure 1. Schema from Project 1

The same key entities from Project 1 — such as Customers, Products, Orders, Sales, Returns, Inventory, and Repair Services — have been preserved in the current implementation. The relational structure and normalization levels used in the original MS Access database were directly translated to the MS SQL Server backend, maintaining data consistency and referential integrity across tables. All core user workflows — such as adding a new product, recording a sale, processing a return, or managing repairs — follow the same logical sequence as defined in the original Access forms.

The familiarity of these workflows helped streamline the transition from Access forms to Java-based interfaces while ensuring that the application remains intuitive to use.

MS SQL SERVER scripts have minor syntax differences from MySQL. Some scripts used for creation of Database in MySQL is shown below:

Creates and uses Database

```
CREATE DATABASE TrentBicycleShopDB;
USE TrentBicycleShopDB;
```

Creates Users Table (For Authentication & Role Management)

```
CREATE TABLE Users (
   UserID INT PRIMARY KEY AUTO_INCREMENT,
   Username VARCHAR(50) UNIQUE NOT NULL,
   HashedPassword VARCHAR(255) NOT NULL,
   Role ENUM('Admin', 'Owner', 'Employee') NOT NULL,
   CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Creates Customer Table

```
CREATE TABLE Customer (
   CustomerID INT PRIMARY KEY AUTO_INCREMENT,
   FirstName VARCHAR(50) NOT NULL,
   LastName VARCHAR(50) NOT NULL,
   Phone VARCHAR(15),
   Email VARCHAR(100) UNIQUE
);
```

Creates Status Table

```
CREATE TABLE Status (
   StatusID INT PRIMARY KEY AUTO_INCREMENT,
   StatusName VARCHAR(50) NOT NULL
);
```

Creates Categories Table

```
CREATE TABLE Categories (
   CategoryID INT PRIMARY KEY AUTO_INCREMENT,
   CategoryName VARCHAR(100) NOT NULL
);
```

Creates Products Table

```
CREATE TABLE Products (
   ProductID INT PRIMARY KEY AUTO_INCREMENT,
   ProductName VARCHAR(100) NOT NULL,
   CategoryID INT,
   Brand VARCHAR(100),
   UnitPrice DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID) ON DELETE SET NULL
);
```

Creates Inventory Table

```
CREATE TABLE Inventory (
   InventoryID INT PRIMARY KEY AUTO_INCREMENT,
   ProductID INT NOT NULL,
   CurrentStock INT NOT NULL DEFAULT 0,
   LastUpdate TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
   FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE
);
```

Creates Sales Table

```
CREATE TABLE Sales (
   SalesID INT PRIMARY KEY AUTO_INCREMENT,
   CustomerID INT NOT NULL,
   SaleDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   TotalAmount DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID) ON DELETE CASCADE
);
```

Set Total Amount to 0

```
ALTER TABLE Sales MODIFY TotalAmount DECIMAL(10,2) NOT NULL DEFAULT 0;
```

Creates SalesDetails Table

```
CREATE TABLE SalesDetails (
   SalesDetailsID INT PRIMARY KEY AUTO_INCREMENT,
   SalesID INT NOT NULL,
   ProductID INT NOT NULL,
   Quantity INT NOT NULL,
   UnitPrice DECIMAL(10,2) NOT NULL,
   SubTotal DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (SalesID) REFERENCES Sales(SalesID) ON DELETE CASCADE,
   FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE
);
```

Creates Returns Table

```
CREATE TABLE Returns (
   ReturnID INT PRIMARY KEY AUTO_INCREMENT,
   SaleID INT NOT NULL,
   ProductID INT NOT NULL,
   ReturnDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   RefundAmount DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (SaleID) REFERENCES Sales(SalesID) ON DELETE CASCADE,
   FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE
);
```

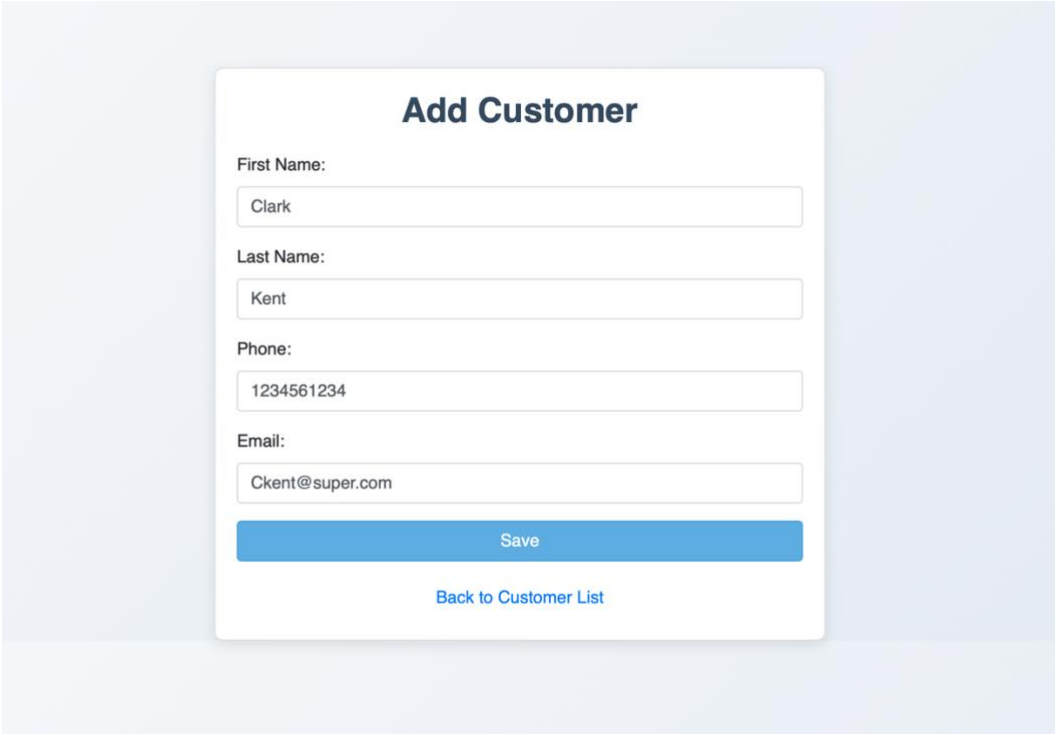
Navigation

To enter application, Username: admin and Password: admin123. Once you enter the app, you can create owner and employee roles. Navigation bar has customers page to products page, to service transaction. You can logout and user newly created users to login for testing access levels.

Manav Patel
239466460

Cosc4606001
Project 2 – Part 2

Forms:



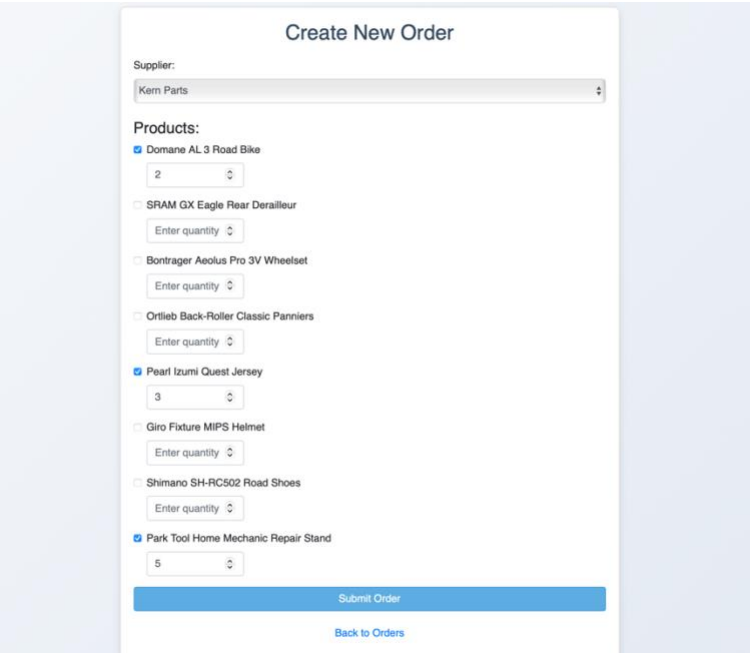
1. New Customer Form

Manav Patel
239466460

Cosc4606001
Project 2 – Part 2

2. New Product Form

3. New Order Form – Receiving Actions and Update in Inventory



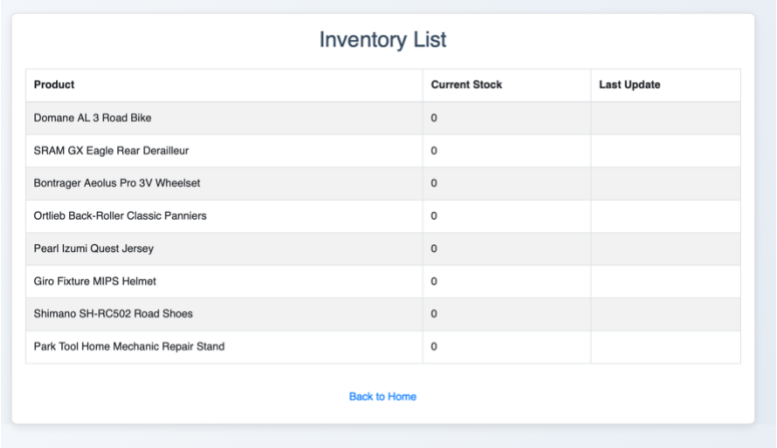
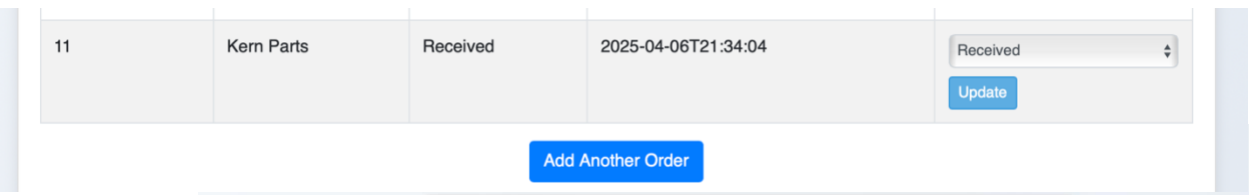
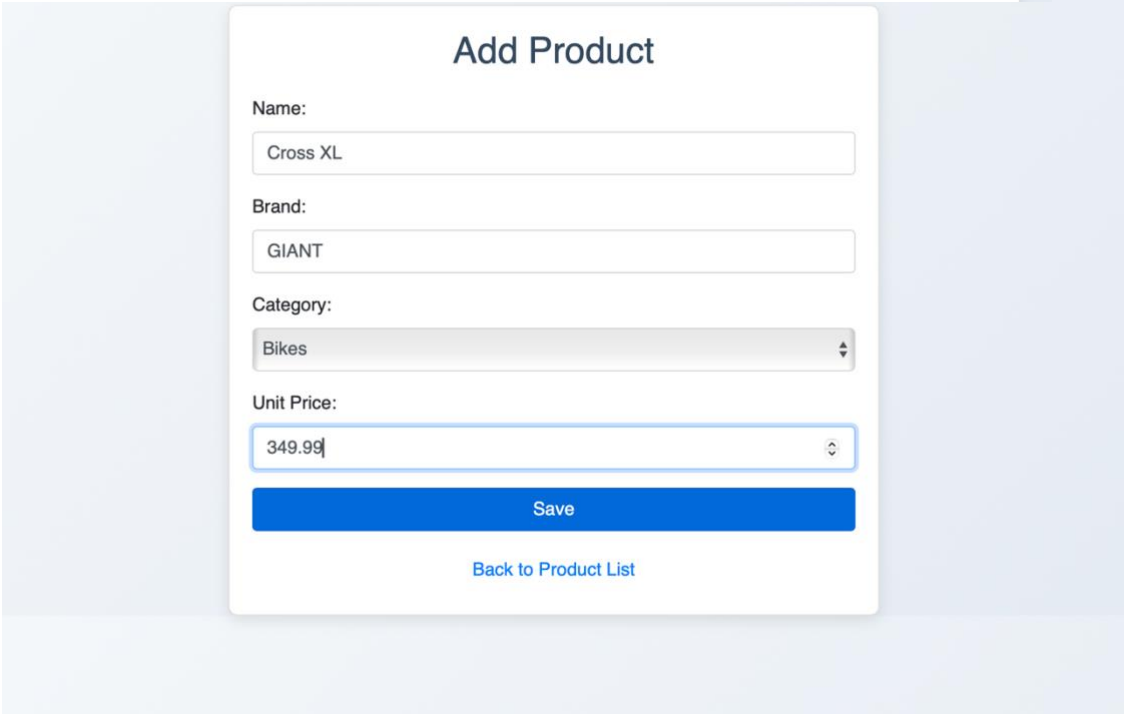


Figure 2.Inventory Before Receiving





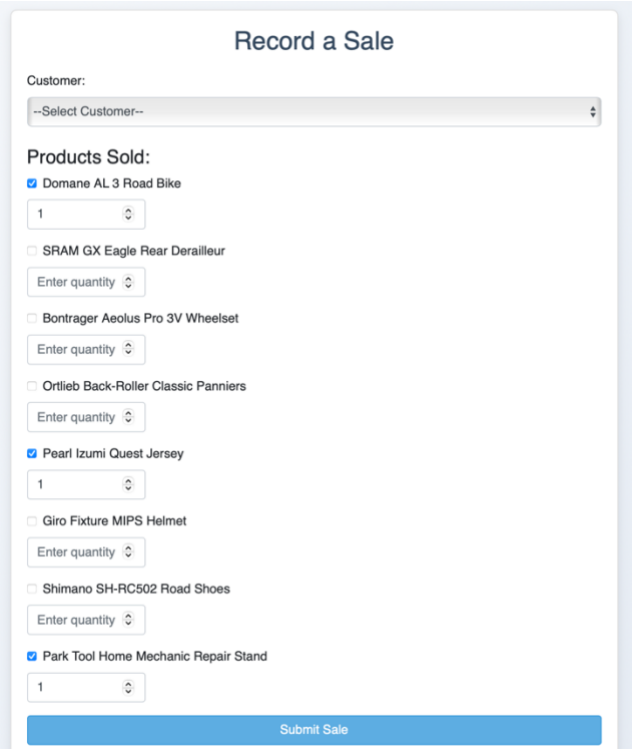
Inventory List

Product	Current Stock	Last Update
Domane AL 3 Road Bike	2	2025-04-06T21:37:56
SRAM GX Eagle Rear Derailleur	0	
Bontrager Aeolus Pro 3V Wheelset	0	
Ortlieb Back-Roller Classic Panniers	0	
Pearl Izumi Quest Jersey	3	2025-04-06T21:37:56
Giro Fixture MIPS Helmet	0	
Shimano SH-RC502 Road Shoes	0	
Park Tool Home Mechanic Repair Stand	5	2025-04-06T21:37:56

Back to Home

Figure 3. Inventory After Receiving and Before Sale

4. Record a sale



Here, we can see that, A customer is buying a bike, a jersey and a repair Stand. After they completed their purchase, the inventory got reduced from 2 to 1 for the bike, 3 to 2 for jersey and repair stand got reduced from 5 to 4.

Inventory List

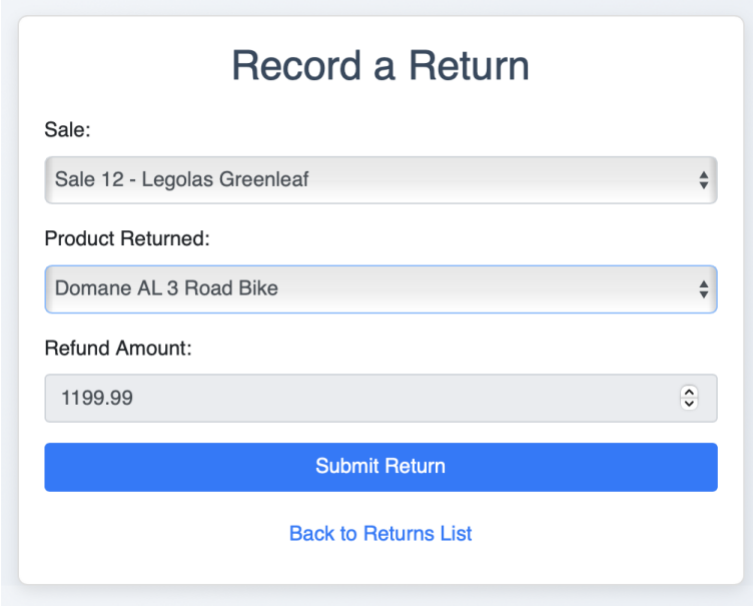
Product	Current Stock	Last Update
Domane AL 3 Road Bike	1	2025-04-06T21:43:21
SRAM GX Eagle Rear Derailleur	0	
Bontrager Aeolus Pro 3V Wheelset	0	
Ortlieb Back-Roller Classic Panniers	0	
Pearl Izumi Quest Jersey	2	2025-04-06T21:43:21
Giro Fixture MIPS Helmet	0	
Shimano SH-RC502 Road Shoes	0	
Park Tool Home Mechanic Repair Stand	4	2025-04-06T21:43:21

Back to Home

Figure 4. After Sale Inventory is Reduced from previous stock

5. Record a return

Here, a customer returns a Bike and the stock is added back into Inventory. It goes from 1 to 2 from previous state.



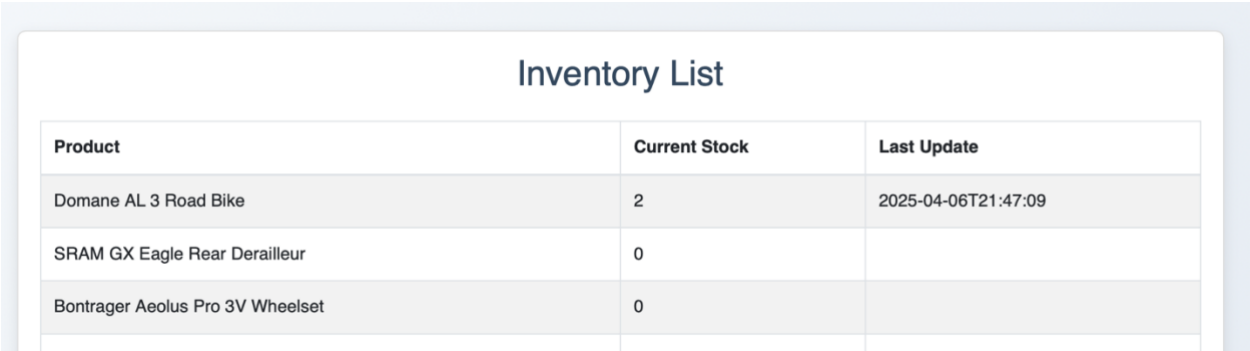


Figure 5. After Return Stock increased for the Road Bike

6. Orders List

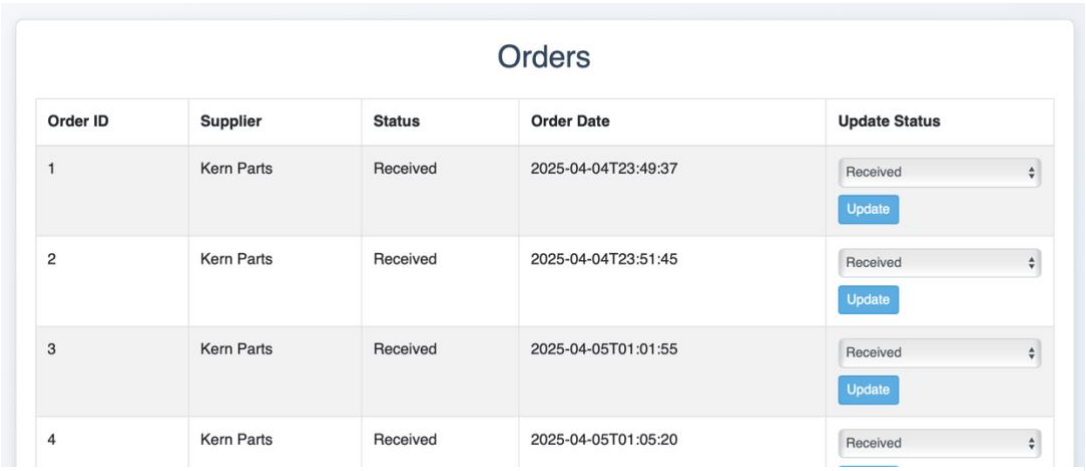
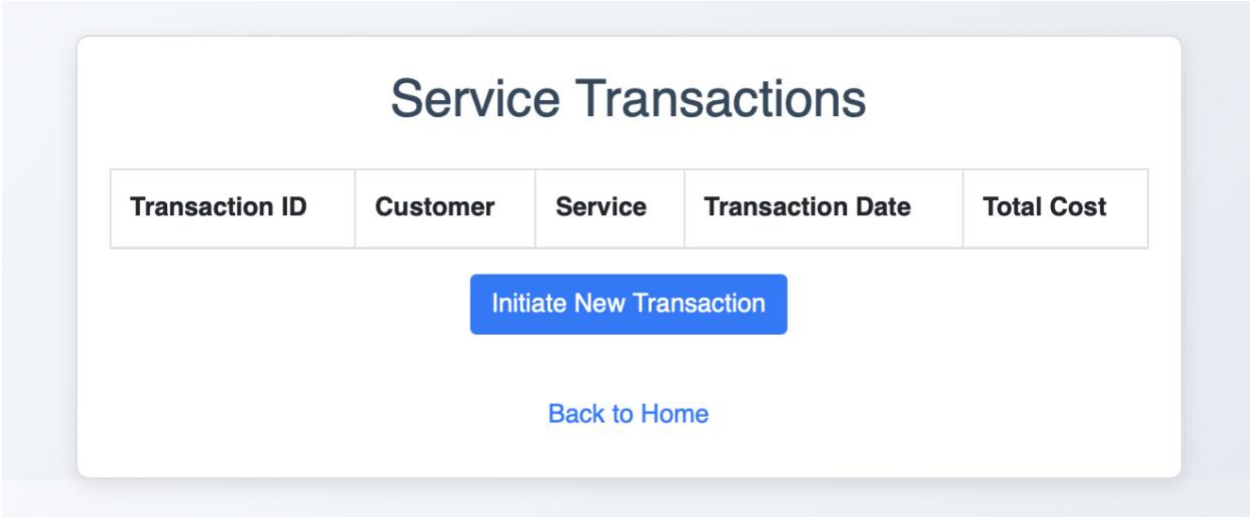


Figure 6. This is order list that can be accessed from navigation bar by the owner

7. Initiate and complete a service transaction – a repair or overhaul



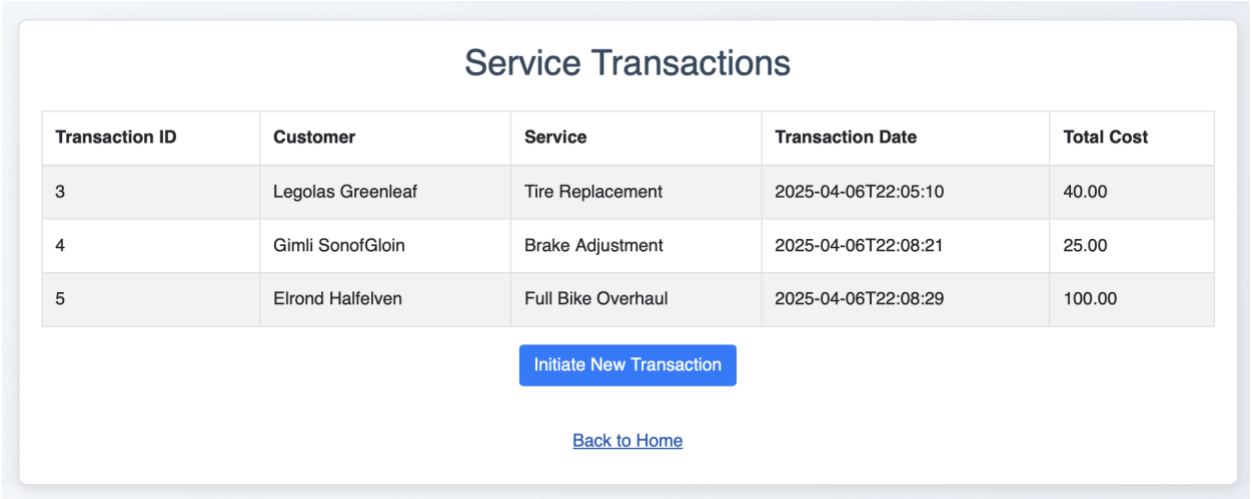


Figure 7. Service Transactions for different services

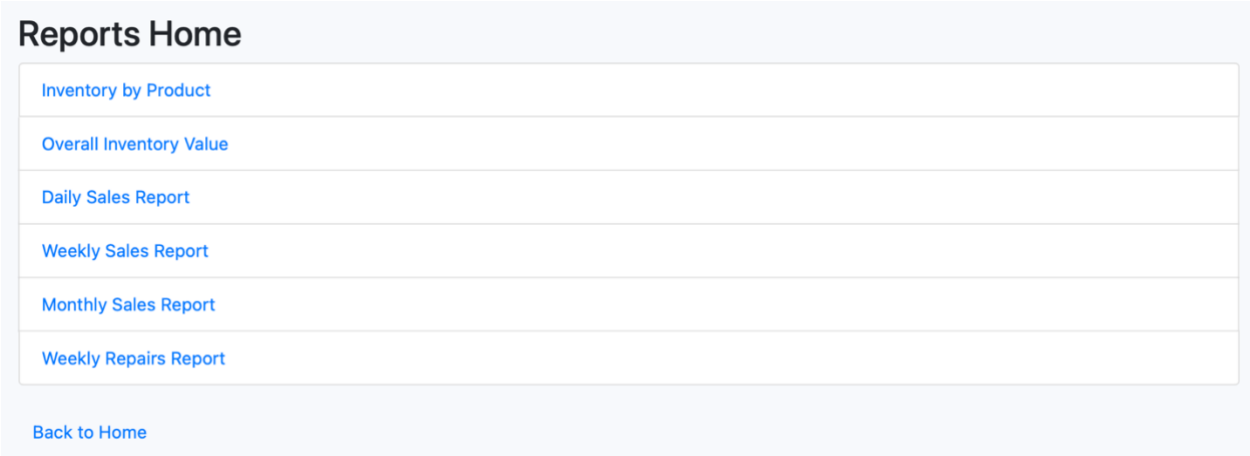
Here, different customers got different services and based on that they were charged accordingly from a simple brake adjustment to a full bike overhaul.

Manav Patel
239466460

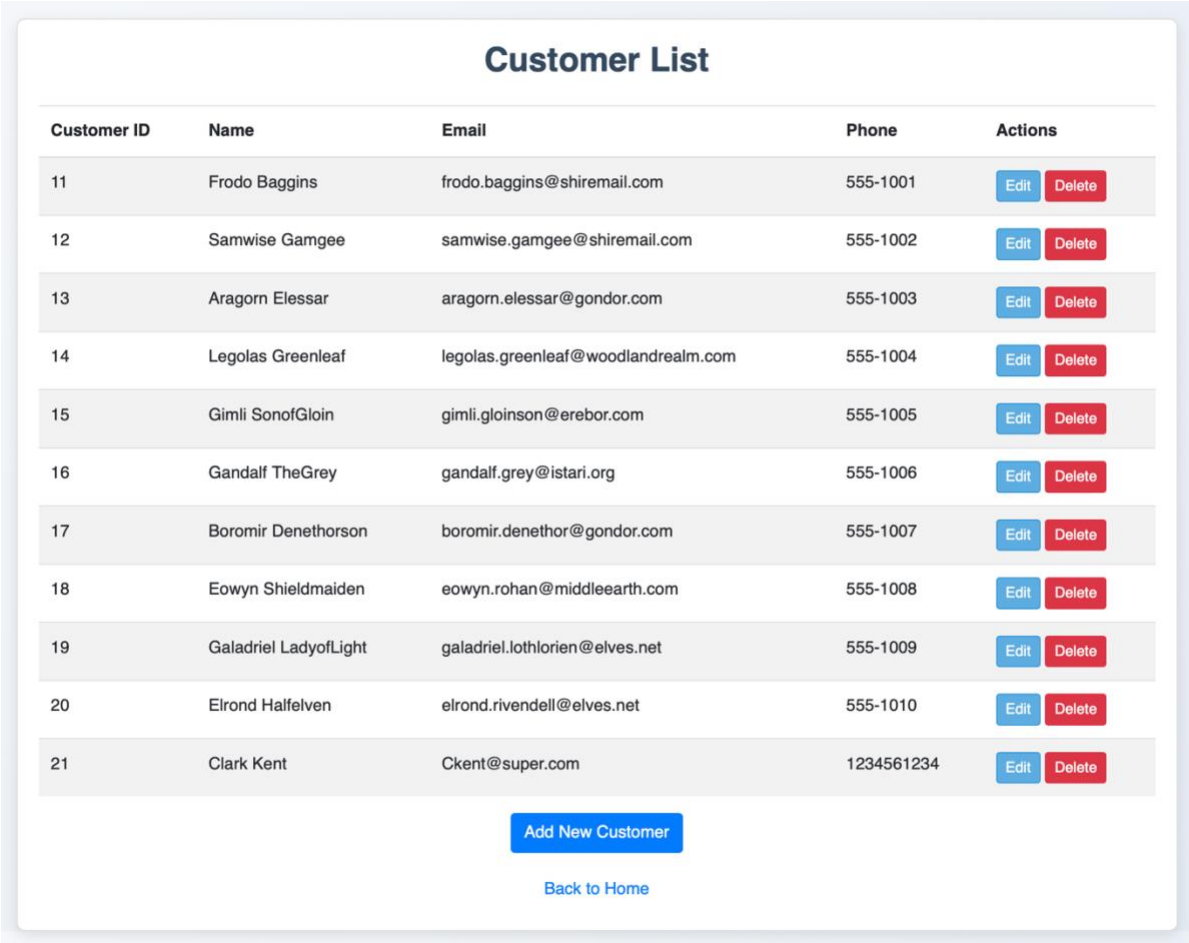
Cosc4606001
Project 2 – Part 2

Reports

1. General report Page:



2. Customer list



3. display a daily, weekly, monthly sales

Daily Sales Report

Sales ID	Customer	Sale Date	Total Amount
12	Legolas Greenleaf	2025-04-06T21:43:21	1494.94
13	Samwise Gamgee	2025-04-06T22:22:10	4313.94
14	Clark Kent	2025-04-06T22:22:18	120.50
15	Boromir Denethorson	2025-04-06T22:22:33	1229.85
16	Elrond Halfelven	2025-04-06T22:23:11	3252.97
17	Elrond Halfelven	2025-04-06T22:23:24	4506.90
18	Aragorn Elessar	2025-04-06T22:23:36	5307.00
19	Eowyn Shieldmaiden	2025-04-06T22:23:52	380.98

Back to Reports Home

Figure 9. Daily Sales

Weekly Sales Report

Sales ID	Customer	Sale Date	Total Amount
12	Legolas Greenleaf	2025-04-06T21:43:21	1494.94
13	Samwise Gamgee	2025-04-06T22:22:10	4313.94
14	Clark Kent	2025-04-06T22:22:18	120.50
15	Boromir Denethorson	2025-04-06T22:22:33	1229.85
16	Elrond Halfelven	2025-04-06T22:23:11	3252.97
17	Elrond Halfelven	2025-04-06T22:23:24	4506.90
18	Aragorn Elessar	2025-04-06T22:23:36	5307.00
19	Eowyn Shieldmaiden	2025-04-06T22:23:52	380.98

Back to Reports Home

Figure 8. Weekly Sales

Monthly Sales Report

Sales ID	Customer	Sale Date	Total Amount
12	Legolas Greenleaf	2025-04-06T21:43:21	1494.94
13	Samwise Gamgee	2025-04-06T22:22:10	4313.94
14	Clark Kent	2025-04-06T22:22:18	120.50
15	Boromir Denethorson	2025-04-06T22:22:33	1229.85
16	Elrond Halfelven	2025-04-06T22:23:11	3252.97
17	Elrond Halfelven	2025-04-06T22:23:24	4506.90
18	Aragorn Elessar	2025-04-06T22:23:36	5307.00
19	Eowyn Shieldmaiden	2025-04-06T22:23:52	380.98

Figure 10. Monthly Sales

4. A table of all inventories of a certain product

Select a Product to View Inventory

Product:

--Select Product--

View Inventory

Back to Reports Home

Figure 12. Form to choose a product

Inventory Details for Domane AL 3 Road Bike

Product ID	Product Name	Current Stock	Last Update
10	Domane AL 3 Road Bike	10	2025-04-06T22:22:10

Back Back to Reports Home

Figure 11. Inventory of a chosen product

5. Total inventory value on hand

Overall Inventory Value

Total Inventory Value: **19051.84**

Product ID	Product Name	Current Stock	Unit Price	Total Value
10	Domane AL 3 Road Bike	10	1199.99	11999.90
11	SRAM GX Eagle Rear Derailleur	2	124.99	249.98
12	Bontrager Aeolus Pro 3V Wheelset	1	1349.00	1349.00
13	Ortlieb Back-Roller Classic Panniers	5	180.00	900.00
14	Pearl Izumi Quest Jersey	11	65.00	715.00
15	Giro Fixture MIPS Helmet	9	75.99	683.91
16	Shimano SH-RC502 Road Shoes	9	120.50	1084.50
18	Park Tool Home Mechanic Repair Stand	9	229.95	2069.55

Back to Reports Home

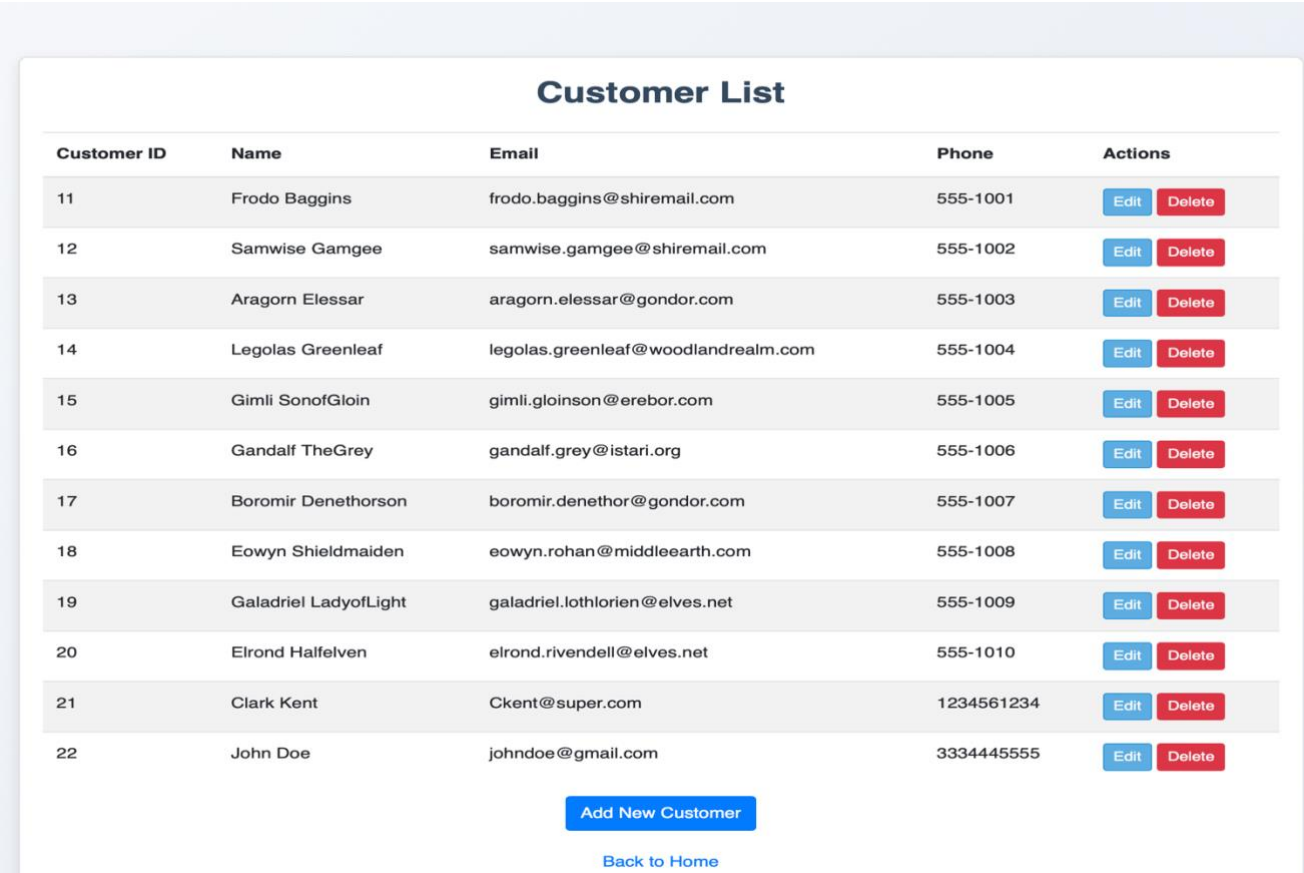
Weekly Repairs Report

Transaction ID	Customer	Service	Transaction Date
3	Legolas Greenleaf	Tire Replacement	2025-04-06T22:05:10
4	Gimli SonofGloin	Brake Adjustment	2025-04-06T22:08:21
5	Elrond Halfelven	Full Bike Overhaul	2025-04-06T22:08:29

Back to Reports Home

6. Weekly repairs

7. Removing a customer



Customer List

Customer ID	Name	Email	Phone	Actions
11	Frodo Baggins	frodo.baggins@shiremail.com	555-1001	Edit Delete
12	Samwise Gamgee	samwise.gamgee@shiremail.com	555-1002	Edit Delete
13	Aragorn Elessar	aragorn.elessar@gondor.com	555-1003	Edit Delete
14	Legolas Greenleaf	legolas.greenleaf@woodlandrealm.com	555-1004	Edit Delete
15	Gimli SonofGloin	gimli.gloinson@erebor.com	555-1005	Edit Delete
16	Gandalf TheGrey	gandalf.grey@istari.org	555-1006	Edit Delete
17	Boromir Denethorson	boromir.denethor@gondor.com	555-1007	Edit Delete
18	Eowyn Shieldmaiden	eowyn.rohan@middleearth.com	555-1008	Edit Delete
19	Galadriel LadyofLight	galadriel.lothlorien@elves.net	555-1009	Edit Delete
20	Elrond Halfelven	elrond.rivendell@elves.net	555-1010	Edit Delete
21	Clark Kent	Ckent@super.com	1234561234	Edit Delete
22	John Doe	johndoe@gmail.com	3334445555	Edit Delete

Add New Customer

Back to Home

Figure 13. We will delete Clark Kent and John Doe

Customer List

Customer ID	Name	Email	Phone	Actions
11	Frodo Baggins	frodo.baggins@shiremail.com	555-1001	Edit Delete
12	Samwise Gamgee	samwise.gamgee@shiremail.com	555-1002	Edit Delete
13	Aragorn Elessar	aragorn.elessar@gondor.com	555-1003	Edit Delete
14	Legolas Greenleaf	legolas.greenleaf@woodlandrealm.com	555-1004	Edit Delete
15	Gimli SonofGloin	gimli.gloinson@erebor.com	555-1005	Edit Delete
16	Gandalf TheGrey	gandalf.grey@istari.org	555-1006	Edit Delete
17	Boromir Denethorson	boromir.denethor@gondor.com	555-1007	Edit Delete
18	Eowyn Shieldmaiden	eowyn.rohan@middleearth.com	555-1008	Edit Delete
19	Galadriel LadyofLight	galadriel.lothlorien@elves.net	555-1009	Edit Delete
20	Elrond Halfelven	elrond.rivendell@elves.net	555-1010	Edit Delete

Add New Customer

Back to Home

Figure 14. Deleted customers, Clark Kent and John Doe

8. Removing a product

Product List

Add New Product

Name	Brand	Category	Unit Price	Actions
Domane AL 3 Road Bike	Trek	1	1199.99	Edit Delete
SRAM GX Eagle Rear Derailleur	SRAM	2	124.99	Edit Delete
Bontrager Aeolus Pro 3V Wheelset	Bontrager	3	1349.00	Edit Delete
Ortlieb Back-Roller Classic Panniers	Ortlieb	4	180.00	Edit Delete
Pearl Izumi Quest Jersey	Pearl Izumi	5	65.00	Edit Delete
Giro Fixture MIPS Helmet	Giro	6	75.99	Edit Delete
Shimano SH-RC502 Road Shoes	Shimano	7	120.50	Edit Delete
Park Tool Home Mechanic Repair Stand	Park Tool	4	229.95	Edit Delete

Figure 15. Deleting Road shoes and Repair Stand (last two products)

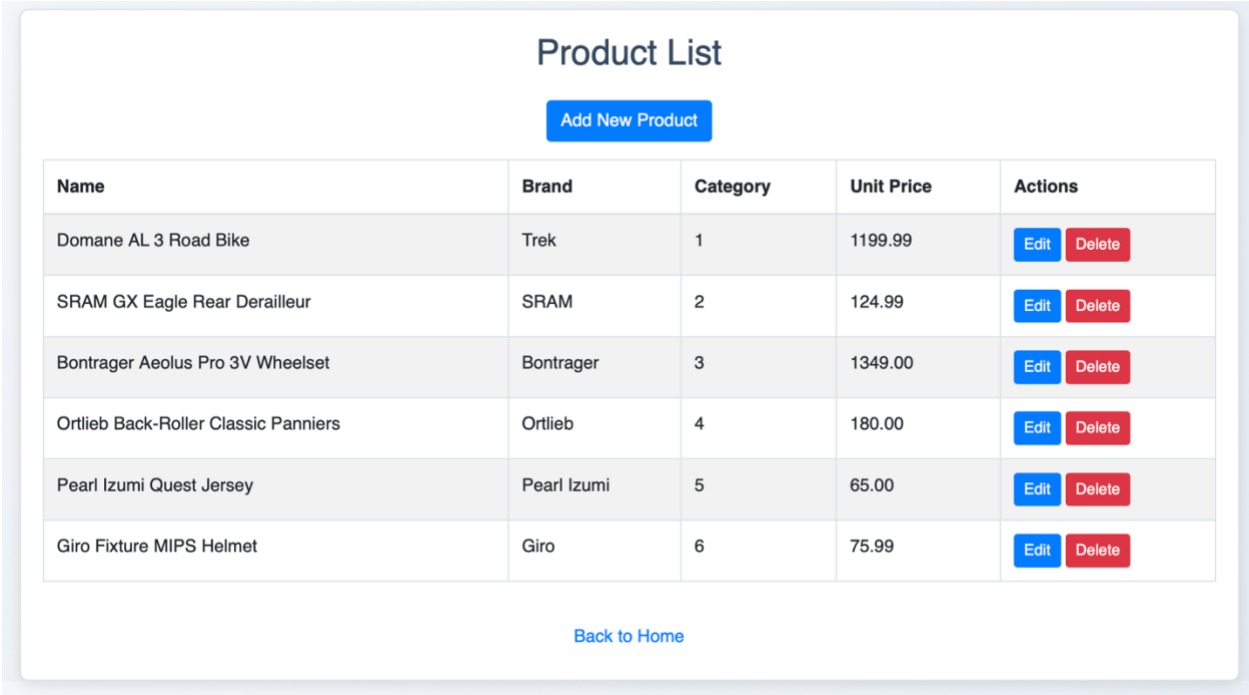


Figure 16. Deleted products, shoes and repair stand

9. Adjusting inventory: This action is performed when a sale, return or order is completed. Additionally, Inventory can be accessed via http://localhost:8080/inventory.

Backend Code:

The backend for this project was developed in a way that allows it to work with both MySQL and Microsoft SQL Server without needing to change the core code. This is possible because of the technologies used, mainly Spring Boot along with JPA (Java Persistence API). JPA acts as a bridge between Java and the database, allowing us to write object-based code instead of raw SQL. As a result, the same set of Java classes and logic can interact with different types of databases just by changing the configuration settings in the 'application.properties' file.

The backend is organized into different layers: models, repositories, services, and controllers. The model layer includes all the entity classes that represent the database tables. These classes are set with JPA annotations so that JPA knows how to map them to the correct tables and columns. The repository layer uses Spring Data JPA to handle common operations like saving, finding, or deleting records. This saves time because we don't have to write the SQL queries

ourselves—Spring takes care of that. Service layer, which contains the main logic for how data should be handled. This includes things like validating inputs or applying business logic before they go to the database. Finally, the controller layer is what works with user actions. It waits for requests (like clicking a button or submitting a form), sends them to the service layer, and returns the results.

Overall, this structure makes the project more organized and easier to maintain. The best part is that switching between MySQL and SQL Server only requires updating a few lines in the configuration file while rest of the code stays the same. Initially I created a small code snippet using JSP and Java with Apache Tomcat but after looking for structure methodology for the folder, I saw a lot of projects developed using Spring Boot. I created a demo app for customer list to check feasibility of creating this project in MySQL and MS-SQL and it worked better than expected which led me to create this whole project in Spring Boot itself.

One key notable issue that took about 5 hours to resolve was Hibernate engine converting the table names into 'snake_case' from my designed 'PascalCase' dueto its naming strategy. I understood the error that the tables were not found and my table names (from backend) were converted in different format but error did not provide me with what part of project causing it. I realized it may be related to JPA so I dived deeper and found a way to stop Hibernate from converting my table names into 'snake_case' using 'spring.jpa.hibernate.naming.physical-strategy'.

Differences in Databases:

During the initial setup, I experimented with connecting the Spring Boot application to both MySQL and Microsoft SQL Server to test compatibility. While MySQL integration was straightforward, setting up MS SQL Server required a bit more effort. One interesting discovery was that although many online resources recommended explicitly including the version number for the Microsoft JDBC driver in the Maven dependency, I found that in recent versions of Spring Boot and Maven, the driver version can be resolved automatically. Simply adding the com.microsoft.sqlserver:mssql-jdbc dependency without specifying a version worked perfectly, with help of improved dependency management in modern Spring Boot projects.

However, integrating with MS SQL Server wasn't as smooth as expected. Unlike MySQL, which allows easier user creation and remote access out of installation process, SQL Server needed some manual configuration. By default, there were no users set up with permission to connect via JDBC, and TCP/IP connections were also disabled. I had to manually enable TCP/IP through the SQL Server Configuration Manager, restart the SQL Server service, and create a login account with the necessary privileges. These steps were not documented clearly in most tutorials, so I had to rely on a combination of official documentation and community forums to get the connection working. I have additionally provided steps for this issue in my installation processes as well.

Once connected, both databases worked well with the backend. The abstraction provided by JPA ensured that my entity classes, services, and repositories remained unchanged regardless of which database was being used. This reinforced the benefit of designing the system in a database-agnostic way, making the backend robust and flexible across different environments.

Security Model:

For this project, I implemented a role-based security system along with encrypted password storage to protect user data and control access to different parts of the application. This was a major improvement over Project 1, which didn't include any kind of login system or security. Spring Boot, along with Spring Security, made it much easier to build a proper security model without having to code everything manually.

To control access, I defined different user roles: Admin, Owner, and Employee. Each role has specific permissions based on what actions they're allowed to perform in the system. For example, only Admins and Owners can access the admin dashboard, only the Owner can manage suppliers and place orders, and all roles (Admin, Owner, Employee) can view and manage customers, products, sales, returns, and service transactions. I set this up in the Spring Security configuration using .authorizeHttpRequests() where I matched URL paths to roles. Public pages like the login screen and static resources (CSS/JS) were left open, while everything else requires the user to be logged in with the appropriate role. This helped keep the application secure while still allowing each type of user to access what they need. Here is how I setup the restriction:

```
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/css/**",
"/js/**").permitAll()
                .requestMatchers("/admin/**").hasAnyRole("Admin", "Owner")
                .requestMatchers("/suppliers/**").hasRole("Owner")
                .requestMatchers("/customers/**", "/products/**",
"/sales/**", "/returns/**",
"/serviceTransactions/**").hasAnyRole("Employee", "Owner", "Admin")
                .requestMatchers("/orders/**").hasRole("Owner")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
```

```
                .loginPage("/login")
                .defaultSuccessUrl("/home", true)
                .permitAll()
            )
            .logout(logout -> logout
                .permitAll()
            );
        return http.build();
    }
```

Additionally, I also made sure that passwords were handled securely. I used BCryptPasswordEncoder, which is provided by Spring Security, to hash passwords before saving them in the database. This means no plain-text passwords are stored, which is really important in case the database is ever exposed. When a new user is created or registered, their password is encoded with BCrypt and only the hashed version is saved. During login, Spring Security compares the entered password to the hash automatically. The best part is that Spring takes care of most of this behind the scenes, so I didn't need to write any custom hashing or comparison logic. I used this to implement encryption:

```
    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new
DaoAuthenticationProvider();
        authProvider.setUserDetailsService(customUserDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
```

Work Log:

Activity	Hours Spent	Dates
Creating Databases in Mysql and MS SQL Server	4	15/3/2025

Created a sample project to test difficulty using JSP, Java and Apache Tomcat	8	19/3/2025 20/3/2025
Created Models of entities	2	21/3/2025
Used Spring Boot to for a sample project for better modularity than before	15	22/3/2025 25/3/2025 26/3/2025
Created Customer, Products and Repairs related service, controller, and repository files	5	28/3/2025
Created the business logic for inventory, orders, sales, and returns model.	12	30/3/2025 1/4/2025
Added CSS to plain HTML files for better UI	3	3/4/2025
Worked on data initializer and security configuration	4	4/4/2025
Test files on different devices	2	5/4/2025
Confirmed steps to install and run applications and created report	8	6/4/2025